

UI controller using **Xcode** for iOS development and working with Swift.

1. Label (UILabel)

Purpose: Displays static or dynamic text.

```
let label = UILabel()
label.text = "Hello, World!"
label.textAlignment = .center
label.font = UIFont.systemFont(ofSize: 18)
label.frame = CGRect(x: 20, y: 50, width: 300, height: 50)
self.view.addSubview(label)
```

2. Button (UIButton)

Purpose: Triggers actions when tapped.

```
let button = UIButton(type: .system)
button.setTitle("Tap Me", for: .normal)
button.frame = CGRect(x: 20, y: 120, width: 150, height: 50)
button.addTarget(self, action: #selector(buttonTapped), for: .touchUpInside)
self.view.addSubview(button)
```

```
@objc func buttonTapped() {
    print("Button was tapped!")
}
```

3. Text Field (UITextField)

Purpose: Accepts user input.

```
let textField = UITextField()
textField.placeholder = "Enter your name"
textField.borderStyle = .roundedRect
textField.frame = CGRect(x: 20, y: 200, width: 300, height: 40)
self.view.addSubview(textField)
```

4. Slider (UISlider)

Purpose: Adjusts a value within a range.

```
let slider = UISlider()
slider.minimumValue = 0
slider.maximumValue = 100
slider.value = 50
slider.frame = CGRect(x: 20, y: 260, width: 300, height: 40)
slider.addTarget(self, action: #selector(sliderChanged), for: .valueChanged)
self.view.addSubview(slider)
```

```
@objc func sliderChanged(_ sender: UISlider) {
    print("Slider value: \(sender.value)")
}
```

5. Switch (UISwitch)

Purpose: Represents an on/off state.

```
let switchControl = UISwitch()
switchControl.frame = CGRect(x: 20, y: 320, width: 0, height: 0)
switchControl.addTarget(self, action: #selector(switchChanged), for: .valueChanged)
self.view.addSubview(switchControl)
```

```
@objc func switchChanged(_ sender: UISwitch) {
    print("Switch is \(sender.isOn ? "On" : "Off")")
}
```

6. Progress View (UIProgressView)

Purpose: Shows progress for a task.

```
let progressView = UIProgressView(progressViewStyle: .default)
progressView.progress = 0.5
progressView.frame = CGRect(x: 20, y: 380, width: 300, height: 10)
self.view.addSubview(progressView)
```

7. Page Control (UIPageControl)

Purpose: Indicates the current page in a paginated interface.

```
let pageControl = UIPageControl()
pageControl.numberOfPages = 5
pageControl.currentPage = 2
pageControl.frame = CGRect(x: 20, y: 420, width: 300, height: 40)
self.view.addSubview(pageControl)
```

8. Table View (UITableView)

Purpose: Displays a scrollable list of items.

```
let tableView = UITableView()
tableView.frame = self.view.bounds
tableView.dataSource = self
tableView.delegate = self
self.view.addSubview(tableView)
```

// Implement UITableViewDataSource methods

```
func tableView(_ tableView: UITableView, numberOfRowsInSection section: Int) -> Int {
    return 10
}
```

```
func tableView(_ tableView: UITableView, cellForRowAt indexPath: IndexPath) ->
UITableViewCell {
    let cell = UITableViewCell(style: .default, reuseIdentifier: "cell")
```

```

        cell.textLabel?.text = "Row \(indexPath.row)"
        return cell
    }

```

9. Collection View (UICollectionView)

Purpose: Displays grid-like content.

```

let layout = UICollectionViewFlowLayout()
let collectionView = UICollectionView(frame: self.view.bounds, collectionViewLayout:
layout)
collectionView.register(UICollectionViewCell.self, forCellWithReuseIdentifier: "cell")
collectionView.dataSource = self
self.view.addSubview(collectionView)

```

// Implement UICollectionViewDataSource

```

func collectionView(_ collectionView: UICollectionView, numberOfItemsInSection section:
Int) -> Int {
    return 20
}

```

```

func collectionView(_ collectionView: UICollectionView, cellForItemAt indexPath: IndexPath)
-> UICollectionViewCell {
    let cell = collectionView.dequeueReusableCell(withReuseIdentifier: "cell", for: indexPath)
    cell.backgroundColor = .blue
    return cell
}

```

10. Image View (UIImageView)

Purpose: Displays an image.

```

let imageView = UIImageView()
imageView.image = UIImage(named: "example")
imageView.contentMode = .scaleAspectFit
imageView.frame = CGRect(x: 20, y: 500, width: 300, height: 200)
self.view.addSubview(imageView)

```

11. Text View (UITextView)

Purpose: Accepts multi-line text input.

```

let textView = UITextView()
textView.text = "This is a text view."
textView.font = UIFont.systemFont(ofSize: 16)
textView.frame = CGRect(x: 20, y: 720, width: 300, height: 100)
self.view.addSubview(textView)

```

12. Web View (WKWebView)

Purpose: Displays web content.

```
import WebKit
```

```
let webView = WKWebView(frame: self.view.bounds)
let url = URL(string: "https://www.apple.com")!
webView.load(URLRequest(url: url))
self.view.addSubview(webView)
```

13. Map View (MKMapView)

Purpose: Displays a map.

```
import MapKit
```

```
let mapView = MKMapView(frame: self.view.bounds)
mapView.mapType = .standard
self.view.addSubview(mapView)
```

14. Date Picker (UIDatePicker)

Purpose: Allows date and time selection.

```
let datePicker = UIDatePicker()
datePicker.datePickerMode = .dateAndTime
datePicker.frame = CGRect(x: 20, y: 850, width: 300, height: 100)
self.view.addSubview(datePicker)
```

15. Picker View (UIPickerView)

Purpose: Displays multiple options for selection.

```
let pickerView = UIPickerView()
pickerView.frame = CGRect(x: 20, y: 970, width: 300, height: 100)
pickerView.dataSource = self
pickerView.delegate = self
self.view.addSubview(pickerView)
```

```
// Implement UIPickerViewDataSource and UIPickerViewDelegate methods
```

```
func numberOfComponents(in pickerView: UIPickerView) -> Int {
    return 1
}
```

```
func pickerView(_ pickerView: UIPickerView, numberOfRowsInComponent component: Int)
-> Int {
    return 5
}
```

```
func pickerView(_ pickerView: UIPickerView, titleForRow row: Int, forComponent
component: Int) -> String? {
    return "Option \(row)"
}
```

```
}
```

16. Search Bar (UISearchBar)

Purpose: Provides a text input field for searches.

```
let searchBar = UISearchBar()
searchBar.placeholder = "Search here"
searchBar.frame = CGRect(x: 0, y: 100, width: self.view.frame.width, height: 50)
searchBar.delegate = self
self.view.addSubview(searchBar)
```

```
// Implement UISearchBarDelegate methods
func searchBarSearchButtonClicked(_ searchBar: UISearchBar) {
    print("Search query: \(searchBar.text ?? "")")
    searchBar.resignFirstResponder()
}
```

17. Gestures (UIGestureRecognizer)

Purpose: Detects gestures like taps, swipes, or pinches.

```
let tapGesture = UITapGestureRecognizer(target: self, action: #selector(viewTapped))
self.view.addGestureRecognizer(tapGesture)
```

```
@objc func viewTapped() {
    print("View was tapped!")
}
```

18. Push Notification

Purpose: Sends notifications to users.

To implement push notifications, follow these steps:

1. Register for notifications in AppDelegate.

import UserNotifications

```
func application(_ application: UIApplication, didFinishLaunchingWithOptions
launchOptions: [UIApplication.LaunchOptionsKey: Any]?) -> Bool {
    UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .badge,
.sound]) { granted, error in
        if granted {
            print("Push notifications permission granted")
        }
    }
    application.registerForRemoteNotifications()
    return true
}
```

2. Handle the notification in AppDelegate.

```
func application(_ application: UIApplication,
didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data) {
    print("Device token: \(deviceToken)")
}
```

19. Image Picker

Purpose: Selects images from the device gallery or camera.

import PhotosUI

```
let imagePicker = UIImagePickerController()
imagePicker.sourceType = .photoLibrary
imagePicker.delegate = self
self.present(imagePicker, animated: true)
```

```
// Implement UIImagePickerControllerDelegate
func imagePickerController(_ picker: UIImagePickerController,
didFinishPickingMediaWithInfo info: [UIImagePickerController.InfoKey : Any]) {
    if let image = info[.originalImage] as? UIImage {
        print("Image selected")
    }
    picker.dismiss(animated: true)
}
```

20. QR Code Scanner

Purpose: Scans QR codes using the camera.

import AVFoundation

```
let captureSession = AVCaptureSession()
guard let videoCaptureDevice = AVCaptureDevice.default(for: .video) else { return }
let videoInput = try? AVCaptureDeviceInput(device: videoCaptureDevice)
captureSession.addInput(videoInput!)
```

```
let metadataOutput = AVCaptureMetadataOutput()
captureSession.addOutput(metadataOutput)
metadataOutput.setMetadataObjectsDelegate(self, queue: DispatchQueue.main)
metadataOutput.metadataObjectTypes = [.qr]
```

```
let previewLayer = AVCaptureVideoPreviewLayer(session: captureSession)
previewLayer.frame = self.view.layer.bounds
self.view.layer.addSublayer(previewLayer)
```

```
captureSession.startRunning()
```

```
// Implement AVCaptureMetadataOutputObjectsDelegate
```

```
func metadataOutput(_ output: AVCaptureMetadataOutput, didOutput metadataObjects:
[AVMetadataObject], from connection: AVCaptureConnection) {
    if let metadataObject = metadataObjects.first as?
AVMetadataMachineReadableCodeObject {
        print("QR Code: \(metadataObject.stringValue ?? "")")
    }
}
```

21. Audio and Video

Purpose: Plays audio or video content.

Audio Example:

```
import AVFoundation
```

```
var audioPlayer: AVAudioPlayer?
```

```
if let audioURL = Bundle.main.url(forResource: "audioFile", withExtension: "mp3") {
    audioPlayer = try? AVAudioPlayer(contentsOf: audioURL)
    audioPlayer?.play()
}
```

Video Example:

```
import AVKit
```

```
let videoURL = URL(string: "https://www.example.com/video.mp4")!
let player = AVPlayer(url: videoURL)
let playerController = AVPlayerViewController()
playerController.player = player
self.present(playerController, animated: true) {
    player.play()
}
```

22. Accelerometer

Purpose: Detects device motion.

```
import CoreMotion
```

```
let motionManager = CMMotionManager()
if motionManager.isAccelerometerAvailable {
    motionManager.accelerometerUpdateInterval = 0.1
    motionManager.startAccelerometerUpdates(to: OperationQueue.main) { (data, error) in
        if let data = data {
            print("X: \(data.acceleration.x), Y: \(data.acceleration.y), Z: \(data.acceleration.z)")
        }
    }
}
```

23. Location Service

Purpose: Gets the user's location.

```
import CoreLocation
```

```
let locationManager = CLLocationManager()
locationManager.delegate = self
locationManager.requestWhenInUseAuthorization()
locationManager.startUpdatingLocation()

// Implement CLLocationManagerDelegate
func locationManager(_ manager: CLLocationManager, didUpdateLocations locations:
[CLLocation]) {
    if let location = locations.last {
        print("Location: \(location.coordinate.latitude), \(location.coordinate.longitude)")
    }
}
```

24. 3D Touch

Purpose: Detects force presses (deprecated, but similar features exist in context menus).

```
override func touchesMoved(_ touches: Set<UITouch>, with event: UIEvent?) {
    if let touch = touches.first {
        print("Force: \(touch.force), Maximum Possible Force:
\\(touch.maximumPossibleForce)")
    }
}
```

25. Attribute Tracking

Purpose: Tracks user interaction with elements.

Example: Tracking button taps using **Analytics Frameworks** like Firebase or Mixpanel.

```
import FirebaseAnalytics
```

```
Analytics.logEvent("button_tap", parameters: [
    "button_name": "Submit" as NSObject
])
```

26. Making the App Live

Purpose: Submit the app to the App Store.

1. Archive your app using Xcode.
2. Validate and upload it using **Xcode Organizer**.
3. Fill in metadata in **App Store Connect**.
4. Submit for review.

27. Overview of Watches

Purpose: Build apps for Apple Watch.

Use **WatchKit** to create a watchOS app.

import WatchKit

```
class InterfaceController: WKInterfaceController {  
    @IBOutlet weak var label: WKInterfaceLabel!  
  
    @IBAction func buttonTapped() {  
        label.setText("Hello Watch!")  
    }  
}
```